

Задание 1

Цель: Добавить конструкторы, получающие массив FunctionPoint.

```
public ArrayTabulatedFunction(FunctionPoint[] points) {
    if (points == null || points.length < 2) {
        throw new IllegalArgumentException("Массив должен содержать не менее 2
элементов");
    }
    for (int i = 1; i < points.length; i++) {
        if (points[i].getX() <= points[i-1].getX()) {
            throw new IllegalArgumentException("Точки должны быть упорядочены по
возрастанию x");
        }
    }
    this.points = new FunctionPoint[points.length];
    this.pointsCount = points.length;
    for (int i = 0; i < points.length; i++) {
        this.points[i] = new FunctionPoint(points[i]);
    }
}
```

Аналогичный конструктор добавлен в LinkedListTabulatedFunction.

```
public LinkedListTabulatedFunction(FunctionPoint[] points) {
    if (points==null||points.length<2) {
        throw new IllegalArgumentException("Массив должен содержать не менее 2
элементов");
    }
    for (int i=1;i<points.length;i++) {
        if (points[i].getX()<=points[i-1].getX()) {
            throw new IllegalArgumentException("Точки должны быть упорядочены по
возрастанию x");
        }
    }
    initList();
    for (int i=0;i<points.length;i++) {
        FunctionNode node=addNodeToTail();
        node.data = new FunctionPoint(points[i]);
    }
}
```

Задание 2

Цель: Создать интерфейс Function и реорганизовать TabulatedFunction.

```
package functions;
public interface Function {
    double getLeftDomainBorder();
    double getRightDomainBorder();
    double getFunctionValue(double x);
}
```

```
package functions;
public interface TabulatedFunction extends Function {
    int getPointsCount();
    FunctionPoint getPoint(int index) throws FunctionPointIndexOutOfBoundsException;
    //... остальное
}
```

Задание 3

Создать пакет functions.basic с различными функциями.

```
package functions.basic;
import functions.Function;
```

```

public class Exp implements Function {

    @Override
    public double getLeftDomainBorder() {
        return Double.NEGATIVE_INFINITY;
    }

    @Override
    public double getRightDomainBorder() {
        return Double.POSITIVE_INFINITY;
    }

    @Override
    public double getFunctionValue(double x) {
        return Math.exp(x);
    }

}

```

```

package functions.basic;

import functions.Function;

// Класс логарифмической функции с заданным основанием

public class Log implements Function {

    private double base;
    // основание логарифма (должно быть > 0 и != 1)
    public Log(double base) {
        if (base <= 0 || base == 1) {
            throw new IllegalArgumentException("Основание логарифма должно быть положительным
и не равным 1");
        }
        this.base = base;
    }

    @Override
    public double getLeftDomainBorder() {
        return 0; // определен на (0, +infinity)
    }

    @Override
    public double getRightDomainBorder() {
        return Double.POSITIVE_INFINITY;
    }

    @Override
    public double getFunctionValue(double x) {
        if (x <= 0) {
            return Double.NaN;
        }
        // log_base(x) = ln(x) / ln(base)
        return Math.log(x) / Math.log(base);
    }

}

```

```

package functions.basic;

// Класс функции синуса;

public class Sin extends TrigonometricFunction {

    @Override
    public double getFunctionValue(double x) {
        return Math.sin(x);
    }

}

```

Идентично – cos, tan

```

package functions.basic;

import functions.Function;

// Базовый класс для тригонометрических функций

```

```

public abstract class TrigonometricFunction implements Function {
    /*
    Имело бы смысл сделать RealDomainFunction с 00 +-infinity,
    а уже от него TrigonometricFunction включающий RealDomainFunction
    с тригонометрическими особенностями, тк экспонента так же могла бы
    включать RealDomainFunction не нарушая математического смысла
    */

    @Override
    public double getLeftDomainBorder() {
        return Double.NEGATIVE_INFINITY;
    }

    @Override
    public double getRightDomainBorder() {
        return Double.POSITIVE_INFINITY;
    }
}

```

Задание 4

Цель: Создать пакет functions.meta для комбинирования функций.

```

package functions.meta;

import functions.Function;

public class Sum implements Function {

    private Function f1;
    private Function f2;

    public Sum(Function f1, Function f2) {
        this.f1 = f1;
        this.f2 = f2;
    }

    @Override
    public double getLeftDomainBorder() {
        // берем самое правое значение
        return Math.max(f1.getLeftDomainBorder(), f2.getLeftDomainBorder());
    }

    @Override
    public double getRightDomainBorder() {
        // берем самое левое значение
        return Math.min(f1.getRightDomainBorder(), f2.getRightDomainBorder());
    }

    @Override
    public double getFunctionValue(double x) {
        return f1.getFunctionValue(x) + f2.getFunctionValue(x);
    }
}

```

```

package functions.meta;

import functions.Function;

// Класс функции, являющейся композицией двух функций
// f(g(x))

public class Composition implements Function {

    private Function outer; // внешняя функция f
    private Function inner; // внутренняя функция g

    public Composition(Function outer, Function inner) {
        this.outer = outer;
        this.inner = inner;
    }

    @Override
    public double getLeftDomainBorder() {
        return inner.getLeftDomainBorder();
    }
}

```

```

@Override
public double getRightDomainBorder() {
    return inner.getRightDomainBorder();
}

@Override
public double getFunctionValue(double x) {
    return outer.getFunctionValue(inner.getFunctionValue(x));
}
}

```

Аналогично реализованы: Mult, Power, Scale, Shift.

Задание 5

Цель: Создать утилитный класс Functions.

```

package functions;

import functions.meta.*;

public final class Functions {

    // запрет создания объектов вне класса
    private Functions() {
        throw new UnsupportedOperationException("Нельзя создать объект класса Functions");
    }

    public static Function shift(Function f, double shiftX, double shiftY) {
        return new Shift(f, shiftX, shiftY);
    }

    // ...и тд
}

```

Задания 6-7

Цель: Создать класс TabulatedFunctions с методами табулирования и ввода-вывода.

```

package functions;

import java.io.*;

public final class TabulatedFunctions {

    private TabulatedFunctions() {
        throw new UnsupportedOperationException("");
    }

    public static TabulatedFunction tabulate(Function function, double leftX, double rightX, int
pointsCount) {
        if (leftX < function.getLeftDomainBorder() || rightX > function.getRightDomainBorder()) {
            throw new IllegalArgumentException("");
        }

        if (leftX >= rightX) {
            throw new IllegalArgumentException(" ");
        }

        if (pointsCount < 2) {
            throw new IllegalArgumentException("");
        }

        double[] values = new double[pointsCount];
        double step = (rightX - leftX) / (pointsCount - 1);

        for (int i = 0; i < pointsCount; i++) {
            values[i] = function.getFunctionValue(leftX + i * step);
        }

        return new ArrayTabulatedFunction(leftX, rightX, values);
    }
}

```

```

        public static void outputTabulatedFunction(TabulatedFunction function, OutputStream out) throws
IOException {
            DataOutputStream dataOut = new DataOutputStream(out);

            int count = function.getPointsCount();
            dataOut.writeInt(count);

            for (int i = 0; i < count; i++) {
                dataOut.writeDouble(function.getPointX(i));
                dataOut.writeDouble(function.getPointY(i));
            }

            dataOut.flush();
            /*
            поток НЕ закрывает - это задача вызывающего кода,
            который открыл поток и знает, нужно ли его дальше применять
            */
        }

        public static TabulatedFunction inputTabulatedFunction(InputStream in) throws IOException {
            DataInputStream dataIn = new DataInputStream(in);

            int count = dataIn.readInt();

            if (count < 2) {
                throw new IOException("'" + count);
            }

            FunctionPoint[] points = new FunctionPoint[count];

            for (int i = 0; i < count; i++) {
                double x = dataIn.readDouble();
                double y = dataIn.readDouble();
                points[i] = new FunctionPoint(x, y);
            }

            return new ArrayTabulatedFunction(points);
        }

        public static void writeTabulatedFunction(TabulatedFunction function, Writer out) throws
IOException {
            PrintWriter writer = new PrintWriter(out);

            int count = function.getPointsCount();
            writer.print(count);

            for (int i = 0; i < count; i++) {
                writer.print(" " + function.getPointX(i));
                writer.print(" " + function.getPointY(i));
            }

            writer.flush();
        }

        public static TabulatedFunction readTabulatedFunction(Reader in) throws IOException {
            StreamTokenizer tokenizer = new StreamTokenizer(in);

            tokenizer.nextToken();
            int count = (int) tokenizer.nval;

            FunctionPoint[] points = new FunctionPoint[count];

            for (int i = 0; i < count; i++) {
                tokenizer.nextToken();
                double x = tokenizer.nval;

                tokenizer.nextToken();
                double y = tokenizer.nval;

                points[i] = new FunctionPoint(x, y);
            }

            return new ArrayTabulatedFunction(points);
        }
    }

```

}

Задание 8

Тестирование всех компонентов:

=== ТЕСТИРОВАНИЕ ===

--- Тест 1: Sin и Cos

x	sin(x)	cos(x)
0,0	0,000000	1,000000
0,1	0,099833	0,995004
0,2	0,198669	0,980067
0,3	0,295520	0,955336
0,4	0,389418	0,921061
0,5	0,479426	0,877583
0,6	0,564642	0,825336
0,7	0,644218	0,764842
0,8	0,717356	0,696707
0,9	0,783327	0,621610
1,0	0,841471	0,540302
1,1	0,891207	0,453596
1,2	0,932039	0,362358
1,3	0,963558	0,267499
1,4	0,985450	0,169967
1,5	0,997495	0,070737
1,6	0,999574	-0,029200
1,7	0,991665	-0,128844
1,8	0,973848	-0,227202
1,9	0,946300	-0,323290
2,0	0,909297	-0,416147
2,1	0,863209	-0,504846
2,2	0,808496	-0,588501
2,3	0,745705	-0,666276
2,4	0,675463	-0,737394
2,5	0,598472	-0,801144
2,6	0,515501	-0,856889
2,7	0,427380	-0,904072
2,8	0,334988	-0,942222
2,9	0,239249	-0,970958
3,0	0,141120	-0,989992
3,1	0,041581	-0,999135

--- Тест 2: Табулированные Sin и Cos

Сравнение оригинальных и табулированных функций:

x	sin(x)	tabSin(x)	cos(x)	tabCos(x)
0,0	0,000000	0,000000	1,000000	1,000000
0,1	0,099833	0,097982	0,995004	0,982723
0,2	0,198669	0,195963	0,980067	0,965446
0,3	0,295520	0,293945	0,955336	0,948170
0,4	0,389418	0,385907	0,921061	0,914355
0,5	0,479426	0,472070	0,877583	0,864608
0,6	0,564642	0,558234	0,825336	0,814862
0,7	0,644218	0,643982	0,764842	0,764620
0,8	0,717356	0,707935	0,696707	0,688404
0,9	0,783327	0,771888	0,621610	0,612188
1,0	0,841471	0,835841	0,540302	0,535972
1,1	0,891207	0,883993	0,453596	0,450633
1,2	0,932039	0,918022	0,362358	0,357141
1,3	0,963558	0,952051	0,267499	0,263648
1,4	0,985450	0,984808	0,169967	0,169931
1,5	0,997495	0,984808	0,070737	0,070437
1,6	0,999574	0,984808	-0,029200	-0,029056
1,7	0,991665	0,984808	-0,128844	-0,128549
1,8	0,973848	0,966204	-0,227202	-0,224761
1,9	0,946300	0,932175	-0,323290	-0,318254
2,0	0,909297	0,898147	-0,416147	-0,411747
2,1	0,863209	0,862441	-0,504846	-0,504272
2,2	0,808496	0,798488	-0,588501	-0,580488
2,3	0,745705	0,734535	-0,666276	-0,656704

2,4	0,675463	0,670582	-0,737394	-0,732920
2,5	0,598472	0,594072	-0,801144	-0,794171
2,6	0,515501	0,507908	-0,856889	-0,843917
2,7	0,427380	0,421745	-0,904072	-0,893664
2,8	0,334988	0,334698	-0,942222	-0,940984
2,9	0,239249	0,236716	-0,970958	-0,958261
3,0	0,141120	0,138735	-0,989992	-0,975537
3,1	0,041581	0,040753	-0,999135	-0,992814

--- Тест 3: $\sin^2(x) + \cos^2(x)$ с разным числом точек

Количество точек: 5

x	$\sin^2 + \cos^2$	Отклонение от 1
0,0	1,000000	0,000000
0,1	0,934912	0,065088
0,2	0,888816	0,111184
0,3	0,861714	0,138286
0,4	0,853604	0,146396
0,5	0,864487	0,135513
0,6	0,894363	0,105637
0,7	0,943232	0,056768
0,8	0,989312	0,010688
0,9	0,926997	0,073003
1,0	0,883675	0,116325
1,1	0,859345	0,140655
1,2	0,854009	0,145991
1,3	0,867665	0,132335
1,4	0,900315	0,099685
1,5	0,951957	0,048043
1,6	0,979028	0,020972
1,7	0,919487	0,080513
1,8	0,878938	0,121062
1,9	0,857382	0,142618
2,0	0,854819	0,145181
2,1	0,871249	0,128751
2,2	0,906671	0,093329
2,3	0,961086	0,038914
2,4	0,969150	0,030850
2,5	0,912382	0,087618
2,6	0,874606	0,125394
2,7	0,855824	0,144176
2,8	0,856034	0,143966
2,9	0,875237	0,124763
3,0	0,913432	0,086568
3,1	0,970621	0,029379

Максимальная ошибка: 0,146396

Количество точек: 50

x	$\sin^2 + \cos^2$	Отклонение от 1
0,0	1,000000	0,000000
0,1	0,998987	0,001013
0,2	0,999568	0,000432
0,3	0,999105	0,000895
0,4	0,999253	0,000747
0,5	0,999339	0,000661
0,6	0,999055	0,000945
0,7	0,999691	0,000309
0,8	0,998975	0,001025
0,9	0,999852	0,000148
1,0	0,999012	0,000988
1,1	0,999456	0,000544
1,2	0,999166	0,000834
1,3	0,999178	0,000822
1,4	0,999437	0,000563
1,5	0,999017	0,000983
1,6	0,999825	0,000175
1,7	0,998974	0,001026
1,8	0,999715	0,000285
1,9	0,999047	0,000953
2,0	0,999357	0,000643
2,1	0,999238	0,000762
2,2	0,999115	0,000885
2,3	0,999546	0,000454
2,4	0,998991	0,001009

2,5	0,999971	0,000029
2,6	0,998984	0,001016
2,7	0,999590	0,000410
2,8	0,999094	0,000906
2,9	0,999268	0,000732
3,0	0,999322	0,000678
3,1	0,999064	0,000936

Максимальная ошибка: 0,001026

Количество точек: 100

x	$\sin^2 + \cos^2$	Отклонение от 1
0,0	1,000000	0,000000
0,1	0,999871	0,000129
0,2	0,999788	0,000212
0,3	0,999750	0,000250
0,4	0,999759	0,000241
0,5	0,999814	0,000186
0,6	0,999916	0,000084
0,7	0,999944	0,000056
0,8	0,999833	0,000167
0,9	0,999768	0,000232
1,0	0,999748	0,000252
1,1	0,999775	0,000225
1,2	0,999848	0,000152
1,3	0,999967	0,000033
1,4	0,999895	0,000105
1,5	0,999802	0,000198
1,6	0,999755	0,000245
1,7	0,999753	0,000247
1,8	0,999798	0,000202
1,9	0,999889	0,000111
2,0	0,999975	0,000025
2,1	0,999854	0,000146
2,2	0,999778	0,000222
2,3	0,999749	0,000251
2,4	0,999765	0,000235
2,5	0,999828	0,000172
2,6	0,999937	0,000063
2,7	0,999922	0,000078
2,8	0,999819	0,000181
2,9	0,999761	0,000239
3,0	0,999750	0,000250
3,1	0,999784	0,000216

Максимальная ошибка: 0,000252

--- Тест 4: Write/Read экспоненты
 Функция записана в файл: exp_tabulated.txt
 Функция прочитана из файла

Сравнение исходной и считанной функции:

x	Исходная	Считанная	Разница
0	1,000000	1,000000	0,000000000
1	2,718282	2,718282	0,000000000
2	7,389056	7,389056	0,000000000
3	20,085537	20,085537	0,000000000
4	54,598150	54,598150	0,000000000
5	148,413159	148,413159	0,000000000
6	403,428793	403,428793	0,000000000
7	1096,633158	1096,633158	0,000000000
8	2980,957987	2980,957987	0,000000000
9	8103,083928	8103,083928	0,000000000
10	22026,465795	22026,465795	0,000000000

--- Тест 5: Output/Input логарифма
 Функция записана в файл: log_tabulated.bin
 Функция прочитана из файла

Сравнение исходной и считанной функции:

x	Исходная	Считанная	Разница
1	0,000000	0,000000	0,000000000
2	0,693147	0,693147	0,000000000
3	1,098612	1,098612	0,000000000
4	1,386294	1,386294	0,000000000
5	1,609438	1,609438	0,000000000

6	1,791759	1,791759	0,000000000
7	1,945910	1,945910	0,000000000
8	2,079442	2,079442	0,000000000
9	2,197225	2,197225	0,000000000
10	2,302585	2,302585	0,000000000

--- Тест 6: Serializable (Array) для $\ln(e^x) = x$

x	Исходная	Считанная
0	0,000000	0,000000
1	1,000000	1,000000
2	2,000000	2,000000
3	3,000000	3,000000
4	4,000000	4,000000
5	5,000000	5,000000
6	6,000000	6,000000
7	7,000000	7,000000
8	8,000000	8,000000
9	9,000000	9,000000
10	10,000000	10,000000

Размер файла: 440 байт

--- Тест 7: Externalizable (LinkedList) для $\ln(e^x) = x$

x	Исходная	Считанная
0	0,000000	0,000000
1	1,000000	1,000000
2	2,000000	2,000000
3	3,000000	3,000000
4	4,000000	4,000000
5	5,000000	5,000000
6	6,000000	6,000000
7	7,000000	7,000000
8	8,000000	8,000000
9	9,000000	9,000000
10	10,000000	10,000000

Размер файла: 241 байт

Задание 9

Реализация сериализации:

Вариант 1: Serializable(реализован в)

```
import java.io.Serializable;
public class ArrayTabulatedFunction implements TabulatedFunction, Serializable {
    ... остальной код
}
```

```
import java.io.Serializable;
public class FunctionPoint implements Serializable{
    ... остальной код
}
```

Вариант 2: Externalizable(реализован в LinkedList)

```
import java.io.*;

public class LinkedListTabulatedFunction implements TabulatedFunction, Externalizable {

    public LinkedListTabulatedFunction() {
        initList();
    }

    @Override public void writeExternal(ObjectOutput out) throws IOException {
        out.writeInt(pointsCount);
        FunctionNode current = head.next;
        while (current != head) {
            out.writeDouble(current.data.getX());
            out.writeDouble(current.data.getY());
            current = current.next;
        }
    }
}
```

```

@Override public void readExternal(ObjectInput in) throws IOException, ClassNotFoundException {
    initList();
    int count = in.readInt();
    for (int i = 0; i < count; i++) {
        double x = in.readDouble();
        double y = in.readDouble();
        addNodeToTail(new FunctionPoint(x, y));
    }
}
... Остальной код
}

```

ТЕСТИРОВАНИЕ СЕРИАЛИЗАЦИИ

```

// Тест 6: Serializable (ArrayTabulatedFunction)

private static void testSerializable() {
    System.out.println("--- Тест 6: Serializable (Array) для ln(e^x) = x");

    String filename = "array_serializable.ser";

    try {
        Function composition = Functions.composition(new Log(Math.E), new Exp());
        TabulatedFunction original = TabulatedFunctions.tabulate(composition, 0, 10, 11);

        try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(filename))) {
            oos.writeObject(original);
        }

        TabulatedFunction restored;
        try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(filename))) {
            restored = (TabulatedFunction) ois.readObject();
        }

        System.out.println("x\tИсходная\tСчитанная");
        for (int x = 0; x <= 10; x++) {
            System.out.printf("%d\t%.6f\t%.6f\n", x, original.getFunctionValue(x),
restored.getFunctionValue(x), x);
        }

        System.out.println("Размер файла: " + new File(filename).length() + " байт\n");

    } catch (IOException | ClassNotFoundException e) {
        System.err.println("Ошибка: " + e.getMessage());
    }
}

// Тест 7: Externalizable (LinkedListTabulatedFunction)
private static void testExternalizable() {
    System.out.println("--- Тест 7: Externalizable (LinkedList) для ln(e^x) = x");

    String filename = "linked_externalizable.ser";

    try {
        Function composition = Functions.composition(new Log(Math.E), new Exp());

        // Externalizable LinkedList с теми же значениями
        double[] values = new double[11];
        for (int i = 0; i <= 10; i++) {
            values[i] = composition.getFunctionValue(i);
        }
        LinkedListTabulatedFunction original = new LinkedListTabulatedFunction(0, 10, values);

        try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(filename))) {
            oos.writeObject(original);
        }

        LinkedListTabulatedFunction restored;
        try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(filename))) {
            restored = (LinkedListTabulatedFunction) ois.readObject();
        }

        System.out.println("x\tИсходная\tСчитанная");
        for (int x = 0; x <= 10; x++) {

```

```
        System.out.printf("%d\t%.6f\t%.6f\n", x, original.getFunctionValue(x),
        restored.getFunctionValue(x), x);
    }

    println("Размер файла: " + new File(filename).length() + " байт\n");

} catch (IOException | ClassNotFoundException e) {
    System.err.println("Ошибка: " + e.getMessage());
}
}
```